

1. Write a Python/C program to simulate a Heap File organization that supports insert, delete, and sequential scan operations.

Algorithm:

1. Maintain a list to represent the file (unordered collection of records).
2. Insert → append record at the end ($O(1)$).
3. Delete → search linearly for the record and remove it.
4. Sequential Scan → traverse the list from start to end and print all records.

Code:

```
class HeapFile:
    def __init__(self):
        self.records = [] # each record: dict with 'id' and other fields

    def insert(self, record):
        self.records.append(record)
        print(f"Inserted: {record}")

    def delete(self, rec_id):
        for r in self.records:
            if r['id'] == rec_id:
                self.records.remove(r)
                print(f"Deleted record with id {rec_id}")
                return
        print(f"Record with id {rec_id} not found")

    def sequential_scan(self):
        print("Sequential Scan of Heap File:")
        for r in self.records:
            print(r)

hf = HeapFile()
hf.insert({'id': 1, 'name': 'Alice'})
hf.insert({'id': 2, 'name': 'Bob'})
hf.insert({'id': 3, 'name': 'Charlie'})
hf.sequential_scan()
hf.delete(2)
hf.sequential_scan()
```

2. Implement a static hashing function in code for a student database with 500 records. Handle collisions using chaining.

Algorithm:

1. Create a hash table of fixed size (e.g., 50 buckets) as a list of lists (chains).
2. Hash function: $\text{bucket} = \text{student_id} \% \text{table_size}$.
3. Insert $\rightarrow$ compute bucket, append record to that bucket's chain.
4. Search $\rightarrow$ compute bucket, scan the chain for matching id.
5. Collisions are handled by chaining (multiple records in the same bucket list).

Code:

```
class StaticHashTable:
    def __init__(self, size=50):
        self.size = size
        self.table = [[] for _ in range(size)]

    def hash_function(self, key):
        return key % self.size

    def insert(self, student_id, name):
        idx = self.hash_function(student_id)
        self.table[idx].append((student_id, name))

    def search(self, student_id):
        idx = self.hash_function(student_id)
        for sid, name in self.table[idx]:
            if sid == student_id:
                return name
        return None

    def show_bucket(self, idx):
        print(f"Bucket {idx}: {self.table[idx]}")

ht = StaticHashTable(size=50)
for i in range(1, 501):
    ht.insert(i, f"Student_{i}")

print("Search student_id=123:", ht.search(123))
print("Search student_id=999:", ht.search(999))
ht.show_bucket(123 % 50)
```

3. Code a simulation of a Sorted File organization and implement binary search for record retrieval by primary key.

Algorithm:

1. Maintain records in a list sorted by primary key.
2. Insert → insert record at the correct position to keep the file sorted.
3. Binary Search → repeatedly divide the sorted list in half to locate the key in $O(\log n)$.

Code:

```
class SortedFile:
    def __init__(self):
        self.records = [] # list of tuples (key, data), kept sorted by key

    def insert(self, key, data):
        pos = 0
        while pos < len(self.records) and self.records[pos][0] < key:
            pos += 1
        self.records.insert(pos, (key, data))

    def binary_search(self, key):
        low, high = 0, len(self.records) - 1
        comparisons = 0
        while low <= high:
            mid = (low + high) // 2
            comparisons += 1
            if self.records[mid][0] == key:
                print(f"Found key {key} in {comparisons} comparisons")
                return self.records[mid]
            elif self.records[mid][0] < key:
                low = mid + 1
            else:
                high = mid - 1
        print(f"Key {key} not found ({comparisons} comparisons)")
        return None

    def display(self):
        print("Sorted File Contents:")
        for r in self.records:
            print(r)
```

```
sf = SortedFile()
for k, v in [(50, "E"), (10, "A"), (30, "C"), (20, "B"), (40, "D")]:
    sf.insert(k, v)
sf.display()
sf.binary_search(30)
sf.binary_search(25)
```

4. Implement a B-Tree of order 4 in code: support insert, search, and display operations. Insert keys: 10, 20, 5, 6, 12, 30, 7, 17.

Algorithm:

1. Order 4 means each node holds at most 3 keys and 4 children.
2. Insert → find leaf position, insert key in sorted order; if node overflows (>3 keys), split it and push the middle key up to parent.
3. Search → traverse from root, comparing key ranges to move to correct child until found or leaf reached.
4. Display → recursively print keys level by level with indentation.

Code:

```
class BTreeNode:
    def __init__(self, leaf=True):
        self.keys = []
        self.children = []
        self.leaf = leaf

class BTree:
    def __init__(self, order=4):
        self.order = order
        self.max_keys = order - 1
        self.root = BTreeNode(leaf=True)

    def search(self, key, node=None):
        node = node or self.root
        i = 0
        while i < len(node.keys) and key > node.keys[i]:
            i += 1
        if i < len(node.keys) and key == node.keys[i]:
            return True
        if node.leaf:
            return False
        return self.search(key, node.children[i])

    def insert(self, key):
        root = self.root
        if len(root.keys) == self.max_keys:
            new_root = BTreeNode(leaf=False)
            new_root.children.append(root)
            self._split_child(new_root, 0)
            self.root = new_root
        self._insert_non_full(self.root, key)
```

```

def _split_child(self, parent, i):
    child = parent.children[i]
    mid = len(child.keys) // 2
    mid_key = child.keys[mid]

    right = BTreeNode(leaf=child.leaf)
    right.keys = child.keys[mid + 1:]
    child.keys = child.keys[:mid]

    if not child.leaf:
        right.children = child.children[mid + 1:]
        child.children = child.children[:mid + 1]

    parent.keys.insert(i, mid_key)
    parent.children.insert(i + 1, right)

def _insert_non_full(self, node, key):
    if node.leaf:
        i = len(node.keys) - 1
        node.keys.append(None)
        while i >= 0 and key < node.keys[i]:
            node.keys[i + 1] = node.keys[i]
            i -= 1
        node.keys[i + 1] = key
    else:
        i = len(node.keys) - 1
        while i >= 0 and key < node.keys[i]:
            i -= 1
        i += 1
        if len(node.children[i].keys) == self.max_keys:
            self._split_child(node, i)
            if key > node.keys[i]:
                i += 1
        self._insert_non_full(node.children[i], key)

```

```
def display(self, node=None, level=0):
    node = node or self.root
    print(" " * level + str(node.keys))
    if not node.leaf:
        for child in node.children:
            self.display(child, level + 1)

bt = BTree(order=4)
for key in [10, 20, 5, 6, 12, 30, 7, 17]:
    bt.insert(key)

print("B-Tree structure:")
bt.display()
print("Search 12:", bt.search(12))
print("Search 99:", bt.search(99))
```

5. Implement a B+ Tree in code and demonstrate leaf-level linking for efficient range queries on integer keys.

Algorithm:

1. Internal nodes store only keys used for routing; actual data lives in leaf nodes.
2. Leaf nodes are linked together via a `next` pointer for fast range scans.
3. Insert $\rightarrow$ descend to correct leaf, insert key in sorted order, split if overflow, propagate a copy of the key up to parent.
4. Range Query $\rightarrow$ find the starting leaf using search, then follow `next` pointers collecting keys until the upper bound is exceeded.

Code:

```
class BPlusNode:
    def __init__(self, leaf=True):
        self.leaf = leaf
        self.keys = []
        self.children = []
        self.next = None

class BPlusTree:
    def __init__(self, order=4):
        self.order = order
        self.root = BPlusNode(leaf=True)

    def insert(self, key):
        self._insert(self.root, key)
        if len(self.root.keys) >= self.order:
            self._split_root()

    def _insert(self, node, key):
        if node.leaf:
            i = 0
            while i < len(node.keys) and node.keys[i] < key:
                i += 1
            node.keys.insert(i, key)
        else:
            i = 0
            while i < len(node.keys) and key >= node.keys[i]:
                i += 1
            self._insert(node.children[i], key)
            if len(node.children[i].keys) >= self.order:
```

```

        self._split_child(node, i)

def _split_child(self, parent, i):
    child = parent.children[i]
    mid = len(child.keys) // 2

    if child.leaf:
        right = BPlusNode(leaf=True)
        right.keys = child.keys[mid:]
        child.keys = child.keys[:mid]
        right.next = child.next
        child.next = right
        up_key = right.keys[0]
    else:
        right = BPlusNode(leaf=False)
        up_key = child.keys[mid]
        right.keys = child.keys[mid + 1:]
        right.children = child.children[mid + 1:]
        child.keys = child.keys[:mid]
        child.children = child.children[:mid + 1]

    parent.keys.insert(i, up_key)
    parent.children.insert(i + 1, right)

def _split_root(self):
    old_root = self.root
    mid = len(old_root.keys) // 2
    new_root = BPlusNode(leaf=False)

    if old_root.leaf:
        right = BPlusNode(leaf=True)
        right.keys = old_root.keys[mid:]
        old_root.keys = old_root.keys[:mid]
        right.next = old_root.next
        old_root.next = right
        new_root.keys = [right.keys[0]]
    else:
        up_key = old_root.keys[mid]
        right = BPlusNode(leaf=False)
        right.keys = old_root.keys[mid + 1:]
        right.children = old_root.children[mid + 1:]

```

```
        old_root.keys = old_root.keys[:mid]
        old_root.children = old_root.children[:mid + 1]
        new_root.keys = [up_key]

    new_root.children = [old_root, right]
    self.root = new_root
```

```
def _find_leaf(self, key):
    node = self.root
    while not node.leaf:
        i = 0
        while i < len(node.keys) and key >= node.keys[i]:
            i += 1
        node = node.children[i]
    return node
```

```
def range_query(self, low, high):
    result = []
    node = self._find_leaf(low)
    while node:
        for k in node.keys:
            if low <= k <= high:
                result.append(k)
            elif k > high:
                return result
        node = node.next
    return result
```

```
bpt = BPlusTree(order=4)
for key in [10, 20, 5, 6, 12, 30, 7, 17, 25, 15]:
    bpt.insert(key)

print("Range query [10, 25]:", bpt.range_query(10, 25))
```

6. Write code to implement a dense primary index on a sorted file. Support search by index key and display the index table.

Algorithm:

1. Store the data file sorted on the primary key.
2. Build a dense index: one index entry (key, record_pointer) for **every** record.
3. Search by key → binary search the index, then use the pointer to fetch the record.
4. Display index → print all (key, pointer) pairs.

Code:

```
class DenseIndex:
    def __init__(self):
        self.data_file = [] # sorted list of records: (key, data)
        self.index = []    # list of (key, pointer)

    def build(self, records):
        self.data_file = sorted(records, key=lambda r: r[0])
        self.index = [(key, pos) for pos, (key, data) in
            enumerate(self.data_file)]

    def search(self, key):
        low, high = 0, len(self.index) - 1
        while low <= high:
            mid = (low + high) // 2
            if self.index[mid][0] == key:
                ptr = self.index[mid][1]
                return self.data_file[ptr]
            elif self.index[mid][0] < key:
                low = mid + 1
            else:
                high = mid - 1
        return None

    def display_index(self):
        print("Dense Index Table (key -> pointer):")
        for key, ptr in self.index:
            print(f" {key} -> {ptr}")
```

```
di = DenseIndex()
di.build([(30, "C"), (10, "A"), (20, "B"), (50, "E"), (40, "D")])
di.display_index()
print("Search 40:", di.search(40))
print("Search 99:", di.search(99))
```

7. Implement extendible hashing in code. Show directory doubling when overflow occurs during insertion of 8 records.

Algorithm:

1. Keep a directory of pointers to buckets, sized $2^{\text{global_depth}}$.
2. Each bucket has a `local_depth` and a fixed capacity.
3. Insert → use the last `global_depth` bits of the key to index into the directory and find the bucket.
4. If the bucket overflows:
 - If `local_depth == global_depth`, double the directory (`global_depth += 1`).
 - Split the bucket, increment its `local_depth`, redistribute its records.
5. Directory doubling lets multiple directory entries point to the same (non-split) bucket.

Code:

```
class Bucket:
    def __init__(self, local_depth, capacity=2):
        self.local_depth = local_depth
        self.capacity = capacity
        self.records = []

    def is_full(self):
        return len(self.records) >= self.capacity

class ExtendibleHashing:
    def __init__(self, capacity=2):
        self.global_depth = 1
        self.capacity = capacity
        self.buckets = [Bucket(1, capacity), Bucket(1, capacity)]
        self.directory = [0, 1]

    def _hash(self, key):
        return key

    def _last_bits(self, key, depth):
        return self._hash(key) & ((1 << depth) - 1)
```

```

def insert(self, key):
    idx = self._last_bits(key, self.global_depth)
    bucket_no = self.directory[idx]
    bucket = self.buckets[bucket_no]
    if key in bucket.records:
        return

    if not bucket.is_full():
        bucket.records.append(key)
        return

    if bucket.local_depth == self.global_depth:
        self._double_directory()

    self._split_bucket(bucket_no)
    self.insert(key)

def _double_directory(self):
    self.directory = self.directory + self.directory
    self.global_depth += 1
    print(f"Directory doubled -> global depth = {self.global_depth}")

def _split_bucket(self, bucket_no):
    old_bucket = self.buckets[bucket_no]
    new_depth = old_bucket.local_depth + 1
    new_bucket = Bucket(new_depth, self.capacity)
    old_bucket.local_depth = new_depth
    self.buckets.append(new_bucket)
    new_bucket_no = len(self.buckets) - 1

    for i in range(len(self.directory)):
        if self.directory[i] == bucket_no:
            if (i >> (new_depth - 1)) & 1:
                self.directory[i] = new_bucket_no

    old_records = old_bucket.records
    old_bucket.records = []
    for key in old_records:
        idx = self._last_bits(key, self.global_depth)
        target_bucket_no = self.directory[idx]
        self.buckets[target_bucket_no].records.append(key)

```

```
def display(self):
    print(f"Global depth: {self.global_depth}")
    print("Directory:", self.directory)
    for i, b in enumerate(self.buckets):
        print(f" Bucket {i} (local depth {b.local_depth}):
{b.records}")
```

```
eh = ExtendibleHashing(capacity=2)
for key in [4, 12, 6, 3, 10, 5, 7, 9]:
    eh.insert(key)
    print(f"After inserting {key}:")
    eh.display()
    print()
```

8. Write a program to simulate secondary storage block access. Calculate the number of I/O operations for sequential vs. indexed retrieval.

Algorithm:

1. Simulate a file stored as blocks, each holding a fixed number of records.
2. Sequential retrieval → read blocks one by one from the start until found (worst case = all blocks).
3. Indexed retrieval → maintain a sparse index (one entry per block, storing block's first key); use it to jump directly to the right block, then scan only within that block.
4. Count and compare the number of block (I/O) accesses for both methods.

Code:

```
class BlockStorage:
    def __init__(self, records, block_size=4):
        self.block_size = block_size
        sorted_records = sorted(records, key=lambda r: r[0])
        self.blocks = [sorted_records[i:i + block_size]
                        for i in range(0, len(sorted_records), block_size)]
        self.index = [(block[0][0], i) for i, block in
                      enumerate(self.blocks)]

    def sequential_search(self, key):
        io_count = 0
        for block in self.blocks:
            io_count += 1
            for rec_key, data in block:
                if rec_key == key:
                    return data, io_count
        return None, io_count

    def indexed_search(self, key):
        io_count = 0
        target_block = 0
        for first_key, block_no in self.index:
            if first_key <= key:
                target_block = block_no
            else:
                break
        io_count += 1
        for rec_key, data in self.blocks[target_block]:
            if rec_key == key:
                return data, io_count
        return None, io_count
```

```
records = [(k, f"data{k}") for k in range(1, 21)]
storage = BlockStorage(records, block_size=4)

key = 17
data, io_seq = storage.sequential_search(key)
print(f"Sequential search for {key}: found={data}, I/O ops={io_seq}")

data, io_idx = storage.indexed_search(key)
print(f"Indexed search for {key}: found={data}, I/O ops={io_idx}")
```

9. Implement a sparse index on a sorted file in code and compare its search performance with a dense index using timing analysis.

Algorithm:

1. Dense index: one entry per record → search is a binary search over all entries.
2. Sparse index: one entry per block (only first key of each block) → locate the block via binary search, then scan linearly within the block.
3. Run many searches on both structures and compare execution time using the `time` module.

Code:

```
import time
import random

class DenseVsSparse:
    def __init__(self, n_records=1000, block_size=10):
        self.records = sorted([(k, f"data{k}") for k in range(1, n_records + 1)])
        self.block_size = block_size
        self.dense_index = [(k, i) for i, (k, v) in enumerate(self.records)]
        self.blocks = [self.records[i:i + block_size]
                        for i in range(0, len(self.records), block_size)]
        self.sparse_index = [(block[0][0], i) for i, block in enumerate(self.blocks)]

    def dense_search(self, key):
        low, high = 0, len(self.dense_index) - 1
        while low <= high:
            mid = (low + high) // 2
            if self.dense_index[mid][0] == key:
                return self.records[self.dense_index[mid][1]]
            elif self.dense_index[mid][0] < key:
                low = mid + 1
            else:
                high = mid - 1
        return None

    def sparse_search(self, key):
        low, high = 0, len(self.sparse_index) - 1
        target_block = 0
        while low <= high:
            mid = (low + high) // 2
```

```
        if self.sparse_index[mid][0] <= key:
            target_block = self.sparse_index[mid][1]
            low = mid + 1
        else:
            high = mid - 1
    for rec_key, data in self.blocks[target_block]:
        if rec_key == key:
            return (rec_key, data)
    return None
```

```
dvs = DenseVsSparse(n_records=1000, block_size=10)
search_keys = [random.randint(1, 1000) for _ in range(200)]
```

```
start = time.perf_counter()
for k in search_keys:
    dvs.dense_search(k)
dense_time = time.perf_counter() - start
```

```
start = time.perf_counter()
for k in search_keys:
    dvs.sparse_search(k)
sparse_time = time.perf_counter() - start
```

```
print(f"Dense index search time: {dense_time:.6f} sec for
{len(search_keys)} lookups")
print(f"Sparse index search time: {sparse_time:.6f} sec for
{len(search_keys)} lookups")
```

10. Write a program to construct and traverse a B+ Tree, then execute a range query for keys between 15 and 45, showing all matching records.

Algorithm:

1. Build the B+ Tree the same way as in Q5 (data in leaves, leaves linked).
2. Traverse → follow the leftmost path down to the first leaf, then walk the next pointers across all leaves to print keys in sorted order.
3. Range Query → locate the leaf containing the lower bound, then follow next pointers collecting keys until a key exceeds the upper bound.

Code:

```
class BPlusNode:
    def __init__(self, leaf=True):
        self.leaf = leaf
        self.keys = []
        self.children = []
        self.next = None

class BPlusTree:
    def __init__(self, order=4):
        self.order = order
        self.root = BPlusNode(leaf=True)

    def insert(self, key):
        self._insert(self.root, key)
        if len(self.root.keys) >= self.order:
            self._split_root()

    def _insert(self, node, key):
        if node.leaf:
            i = 0
            while i < len(node.keys) and node.keys[i] < key:
                i += 1
            node.keys.insert(i, key)
        else:
            i = 0
            while i < len(node.keys) and key >= node.keys[i]:
                i += 1
            self._insert(node.children[i], key)
            if len(node.children[i].keys) >= self.order:
                self._split_child(node, i)
```

```

def _split_child(self, parent, i):
    child = parent.children[i]
    mid = len(child.keys) // 2

    if child.leaf:
        right = BPlusNode(leaf=True)
        right.keys = child.keys[mid:]
        child.keys = child.keys[:mid]
        right.next = child.next
        child.next = right
        up_key = right.keys[0]
    else:
        right = BPlusNode(leaf=False)
        up_key = child.keys[mid]
        right.keys = child.keys[mid + 1:]
        right.children = child.children[mid + 1:]
        child.keys = child.keys[:mid]
        child.children = child.children[:mid + 1]

    parent.keys.insert(i, up_key)
    parent.children.insert(i + 1, right)

def _split_root(self):
    old_root = self.root
    mid = len(old_root.keys) // 2
    new_root = BPlusNode(leaf=False)

    if old_root.leaf:
        right = BPlusNode(leaf=True)
        right.keys = old_root.keys[mid:]
        old_root.keys = old_root.keys[:mid]
        right.next = old_root.next
        old_root.next = right
        new_root.keys = [right.keys[0]]
    else:
        up_key = old_root.keys[mid]
        right = BPlusNode(leaf=False)
        right.keys = old_root.keys[mid + 1:]
        right.children = old_root.children[mid + 1:]
        old_root.keys = old_root.keys[:mid]
        old_root.children = old_root.children[:mid + 1]

```

```

        new_root.keys = [up_key]

    new_root.children = [old_root, right]
    self.root = new_root

def _leftmost_leaf(self):
    node = self.root
    while not node.leaf:
        node = node.children[0]
    return node

def traverse(self):
    node = self._leftmost_leaf()
    result = []
    while node:
        result.extend(node.keys)
        node = node.next
    return result

def _find_leaf(self, key):
    node = self.root
    while not node.leaf:
        i = 0
        while i < len(node.keys) and key >= node.keys[i]:
            i += 1
        node = node.children[i]
    return node

def range_query(self, low, high):
    result = []
    node = self._find_leaf(low)
    while node:
        for k in node.keys:
            if low <= k <= high:
                result.append(k)
            elif k > high:
                return result
        node = node.next
    return result

```

```
keys = [10, 20, 5, 6, 12, 30, 7, 17, 45, 25, 15, 40, 22, 35]
bpt = BPlusTree(order=4)
for k in keys:
    bpt.insert(k)

print("Full traversal (sorted, via leaf links):", bpt.traverse())
print("Range query [15, 45]:", bpt.range_query(15, 45))
```